# Column Types

Copy Page

Browse the full list of monday.com column types, their API support levels, and implementation types for reading and writing column values

monday.com boards support a variety of column types, each designed to store specific kinds of data. The API lets you read, filter, create, update, and clear most column types, though some are read-only or unsupported.

Each column type has a dedicated reference page (linked below) with examples for querying, filtering, and mutating values. The tables on this page show every column type, its implementation type, and the `ColumnType` enum value used in the API.

## Reading column values

Column values are returned through the `column_values` field on items. Each column type maps to a specific GraphQL implementation type (e.g., `StatusValue`, `TextValue`, `DropdownValue`). Use inline fragments to access type-specific fields:

GraphQL

```graphql
query {
  items(ids: [1234567890]) {
    column_values {
      ... on StatusValue {
        label
        index
      }
      ... on TextValue {
        text
        value
      }
    }
  }
}
```

All implementations share a common set of fields from the `ColumnValue` `type`, and `value`. Type-specific fields (like `label` on `StatusValue` or only available through inline fragments.

Need help? Ask AI!

# Supported columns

These column types support read operations through `column_values`, and most support write operations through `change_simple_column_value` or `change_multiple_column_values`. See each column's reference page for details on supported operations.

| Column title | Implementation | Column type |
|---|---|---|
| Button | `ButtonValue` | `button` |
| Checkbox | `CheckboxValue` | `checkbox` |
| Color picker | `ColorPickerValue` | `color_picker` |
| Connect boards | `BoardRelationValue` | `board_relation` |
| Country | `CountryValue` | `country` |
| Date | `DateValue` | `date` |
| Dependency | `DependencyValue` | `dependency` |
| Dropdown | `DropdownValue` | `dropdown` |
| Email | `EmailValue` | `email` |
| Files | `FileValue` | `file` |
| Hour | `HourValue` | `hour` |
| Link | `LinkValue` | `link` |
| Location | `LocationValue` | `location` |
| Long text | `LongTextValue` | `long_text` |
| monday doc | `DocValue` | `doc` |
| Name | | `name` |
| Numbers | `NumbersValue` | `numbers` |
| People | `PeopleValue` | `people` |
| Phone | `PhoneValue` | `phone` |
| Rating | `RatingValue` | `rating` |
| Status | `StatusValue` | `status` |
| Subitems | `SubtasksValue` | `subtasks` |
| Tags | `TagsValue` | `tags` |
| Text | `TextValue` | `text` |

| Column title | Implementation | Column type |
|---|---|---|
| Timeline | `TimelineValue` | `timeline` |
| Time tracking | `TimeTrackingValue` | `time_tracking` |
| Vote | `VoteValue` | `vote` |
| Week | `WeekValue` | `week` |
| World clock | `WorldClockValue` | `world_clock` |

## Read-only columns

These columns return values through the API but cannot be written to. Their data is auto-generated or derived from other columns.

| Column title | Implementation | Column type |
|---|---|---|
| Creation log | `CreationLogValue` | `creation_log` |
| Formula | `FormulaValue` | `formula` |
| Item ID | `ItemIdValue` | `item_id` |
| Last updated | `LastUpdatedValue` | `last_updated` |
| Mirror | `MirrorValue` | `mirror` |
| Progress tracking | `ProgressValue` | `progress` |

## Calculated columns

These columns are computed at render time and are not accessible through the `column_values` API.

| Column title | Column type |
|---|---|
| Auto number | `auto_number` |

## Deprecated columns

These column types have been deprecated. They remain accessible via the API for backwards compatibility, but you should use the recommended replacement.

| Column title | Implementation | Column type | Replacement |
|---|---|---|---|
| Person | `PersonValue` | `person` | People |

| Column title | Implementation | Column type | Replacement |
|---|---|---|---|
| Team | `TeamValue` | `team` | People |

## Other column types

The following column types exist in the API schema but have limited or no direct API support for reading and writing values.

| Column title | Implementation | Column type | Notes |
|---|---|---|---|
| Integration | `IntegrationValue` | `integration` | Managed by integrations; values cannot be set directly. |

## Creating columns

You can create new columns on a board using the generic `create_column` mutation, which works for any column type. Pass the column type as the `column_type` argument.

For **status** and **dropdown** columns, the API also provides typed mutations with strongly typed settings:

- `create_status_column` — create a status column with label and color configuration
- `create_dropdown_column` — create a dropdown column with predefined options

## Updating column values

You can update column values using one of two mutations:

- `change_simple_column_value` — accepts a string value for the column
- `change_multiple_column_values` — accepts a JSON object to update one or more columns at once

The expected format varies by column type. See each column's reference page for the specific value format and examples.

## Filtering by column values

You can filter items by column values using `items_page` with `query_params`. Each column type supports a specific set of filter operators (e.g., `any_of`, `contains_text`, `greater_than`). See each column's reference page for supported operators and compare value formats.

GraphQL

```
query {
  boards(ids: [1234567890]) {
    items_page(
      query_params: {
        rules: [
          {
            column_id: "status"
            compare_value: [1]
            operator: any_of
          }
        ]
      }
    ) {
      items {
        id
        name
      }
    }
  }
}
```

## Column type schema

You can retrieve the JSON schema for any column type's settings using the `get_column_type_schema` query. This returns the structure, validation rules, and available properties for a column's configuration.

GraphQL

```
query {
  get_column_type_schema(
    type: status
  )
}
```

The schema is useful for validating column settings programmatically, building dynamic UIs, or understanding the available configuration options for each column type.

Updated 3 months ago

← Columns Auto Number →

Did this page help you? Yes No